

EJERCICIO 1: Búsqueda de Libros Disponibles (BÁSICO)

Conceptos: `search()`, dominios, Recordsets

Objetivo

Crear un método que busque todos los libros que están disponibles para préstamo (con cantidad disponible > 0).

Descripción del Problema

Los bibliotecarios necesitan una forma rápida de ver qué libros están actualmente disponibles para préstamo, sin tener que revisar manualmente cada registro.

Tareas

1. Crear un método `get_available_books()` en el modelo `library.book`
2. El método debe retornar un recordset con libros donde `available_qty > 0`
3. Agregar un botón "Ver Disponibles" en la vista tree de libros

Solución

Archivo: `models/library_book.py`

python

```
Python
from odoo import models, fields, api

class LibraryBook(models.Model):
    _inherit = 'library.book'

    def get_available_books(self):
        """Retorna todos los libros disponibles para
        préstamo"""
        # Búsqueda con dominio
        available_books = self.search([
            ('available_qty', '>', 0),
            ('state', '=', 'available')
        ])
```

```
# Retornar acción para mostrar resultados
return {
    'name': 'Libros Disponibles',
    'type': 'ir.actions.act_window',
    'res_model': 'library.book',
    'view_mode': 'tree,form',
    'domain': [('id', 'in', available_books.ids)],
    'context': {'search_default_available': 1}
}
```

Archivo: `views/library_book_views.xml`

xml

```
XML
<record id="library_book_view_tree_inherit"
model="ir.ui.view">
    <field name="name">library.book.tree.inherit</field>
    <field name="model">library.book</field>
    <field name="inherit_id"
ref="solt_library.library_book_view_tree"/>
    <field name="arch" type="xml">
        <xpath expr="//tree" position="inside">
            <header>
                <button name="get_available_books"
                    string="Ver Disponibles"
                    type="object"
                    class="btn-primary"/>
            </header>
        </xpath>
    </field>
</record>
```

Verificación

1. Actualizar el módulo
2. Ir a Biblioteca → Libros
3. Hacer clic en "Ver Disponibles"
4. Verificar que solo aparecen libros con `available_qty > 0`

EJERCICIO 2: Campo Calculado - Días de Préstamo Restantes (BÁSICO-INTERMEDIO)

Conceptos: `@api.depends`, campos calculados, `compute`

Objetivo

Agregar un campo calculado que muestre cuántos días faltan para que un préstamo venza.

Descripción del Problema

Los bibliotecarios necesitan ver de un vistazo cuántos días quedan antes de que un préstamo se vuelva vencido, para poder hacer seguimiento proactivo.

Tareas

1. Agregar campo calculado `days_remaining` en `library.loan`
2. El campo debe calcular: `expected_return_date - fecha_actual`
3. Si el préstamo ya venció, mostrar valor negativo
4. Mostrar el campo en la vista tree y form

Solución

Archivo: `models/library_loan.py`

python

```
Python
from odoo import models, fields, api
from datetime import date

class LibraryLoan(models.Model):
    _inherit = 'library.loan'

    days_remaining = fields.Integer(
        string='Días Restantes',
        compute='_compute_days_remaining',
        store=True,
        help='Días restantes hasta la fecha de devolución esperada'
    )
```

```

@api.depends('expected_return_date', 'state')
def _compute_days_remaining(self):
    """Calcula los días restantes para la devolución"""
    today = date.today()

    for loan in self:
        if loan.state in ['returned', 'cancelled']:
            loan.days_remaining = 0
        elif loan.expected_return_date:
            delta = loan.expected_return_date - today
            loan.days_remaining = delta.days
        else:
            loan.days_remaining = 0

```

Archivo: `views/library_loan_views.xml`

xml

```

XML
<record id="library_loan_view_tree_inherit"
model="ir.ui.view">
    <field name="name">library.loan.tree.inherit</field>
    <field name="model">library.loan</field>
    <field name="inherit_id"
ref="solt_library.library_loan_view_tree"/>
    <field name="arch" type="xml">
        <xpath expr="//field[@name='expected_return_date']"
position="after">
            <field name="days_remaining"
                decoration-danger="days_remaining < 0"
                decoration-warning="days_remaining <= 3
and days_remaining >= 0"/>
        </xpath>
    </field>
</record>

<record id="library_loan_view_form_inherit"
model="ir.ui.view">

```

```
<field name="name">library.loan.form.inherit</field>
<field name="model">library.loan</field>
<field name="inherit_id"
ref="solt_library.library_loan_view_form"/>
  <field name="arch" type="xml">
    <xpath expr="//field[@name='expected_return_date']"
position="after">
      <field name="days_remaining"/>
    </xpath>
  </field>
</record>
```

Verificación

1. Actualizar el módulo
2. Ir a Biblioteca → Préstamos
3. Verificar que aparece columna "Días Restantes"
4. Los préstamos próximos a vencer (≤ 3 días) deben aparecer en amarillo
5. Los préstamos vencidos (< 0 días) deben aparecer en rojo

EJERCICIO 3: Filtrar y Mapear Autores Populares (INTERMEDIO)

Conceptos: `filtered()`, `mapped()`, operaciones de Recordsets

Objetivo

Crear un método que identifique autores populares (con más de 3 libros) y retorne una lista con sus nombres.

Descripción del Problema

La biblioteca quiere promocionar a los autores más prolíficos en su colección para crear una sección especial en la interfaz.

Tareas

1. Crear método `get_popular_authors()` en `library.author`
2. Filtrar autores con más de 3 libros
3. Retornar lista de nombres de autores

4. Agregar campo Many2many en un wizard para mostrar resultados

Solución

Archivo: `models/library_author.py`

python

```
Python
from odoo import models, fields, api

class LibraryAuthor(models.Model):
    _inherit = 'library.author'

    book_count = fields.Integer(
        string='Cantidad de Libros',
        compute='_compute_book_count',
        store=True
    )

    @api.depends('book_ids')
    def _compute_book_count(self):
        """Calcula la cantidad de libros del autor"""
        for author in self:
            author.book_count = len(author.book_ids)

    def get_popular_authors(self):
        """
        Retorna autores populares (con más de 3 libros)
        Demuestra uso de filtered() y mapped()
        """
        # Obtener todos los autores
        all_authors = self.search([])

        # Filtrar autores con más de 3 libros
        popular_authors = all_authors.filtered(lambda a:
a.book_count > 3)

        # Mapear para obtener solo los nombres
        author_names = popular_authors.mapped('name')

        # Mostrar en log
```

```

import logging
_logger = logging.getLogger(__name__)
_logger.info(f"Autores populares: {author_names}")

# Retornar acción con los autores filtrados
return {
    'name': 'Autores Populares (>3 libros)',
    'type': 'ir.actions.act_window',
    'res_model': 'library.author',
    'view_mode': 'kanban,tree,form',
    'domain': [('id', 'in', popular_authors.ids)],
    'context': {'default_book_count': 3}
}

```

Archivo: `views/library_author_views.xml`

xml

```

XML
<record id="library_author_view_tree_inherit"
model="ir.ui.view">
    <field name="name">library.author.tree.inherit</field>
    <field name="model">library.author</field>
    <field name="inherit_id"
ref="solt_library.library_author_view_tree"/>
    <field name="arch" type="xml">
        <xpath expr="//tree" position="attributes">
            <attribute name="decoration-success">book_count
            &gt; 3</attribute>
        </xpath>
        <xpath expr="//field[@name='name']" position="after">
            <field name="book_count"/>
        </xpath>
        <xpath expr="//tree" position="inside">
            <header>
                <button name="get_popular_authors"
                    string="Ver Autores Populares"
                    type="object"
                    class="btn-success"/>
            </header>
        </xpath>
    </field>
</record>

```

```
        </header>
    </xpath>
</field>
</record>
```

Verificación

1. Actualizar el módulo
2. Ir a Biblioteca → Autores
3. Verificar que aparece columna "Cantidad de Libros"
4. Hacer clic en "Ver Autores Populares"
5. Verificar que solo aparecen autores con más de 3 libros (resaltados en verde)

EJERCICIO 4: Onchange - Sugerir Fecha de Devolución (INTERMEDIO)

Conceptos: `@api.onchange`, actualización dinámica de UI

Objetivo

Al seleccionar un miembro en un préstamo, calcular automáticamente la fecha de devolución esperada según su tipo de membresía.

Descripción del Problema

Los bibliotecarios tienen que calcular manualmente la fecha de devolución basándose en el tipo de membresía del usuario. Esto debe automatizarse para evitar errores.

Tareas

1. Crear `onchange` en campo `member_id` del modelo `library_loan`
2. Calcular `expected_return_date` = fecha_actual + días_de_préstamo del miembro
3. Mostrar advertencia si el miembro tiene préstamos vencidos
4. Verificar que funciona en tiempo real en la UI

Solución

Archivo: `models/library_loan.py`

python

Python

```
from odoo import models, fields, api
from datetime import date, timedelta

class LibraryLoan(models.Model):
    _inherit = 'library.loan'

    @api.onchange('member_id', 'loan_date')
    def _onchange_member_suggest_return_date(self):
        """
        Cuando se selecciona un miembro, sugerir fecha de
        devolución
        basada en su tipo de membresía
        """
        if self.member_id and self.loan_date:
            # Obtener días de préstamo según tipo de membresía
            loan_days = self.member_id.loan_days or 14 #
            Default 14 días

            # Calcular fecha esperada de devolución
            self.expected_return_date = self.loan_date +
            timedelta(days=loan_days)

            # Verificar si el miembro tiene préstamos vencidos
            if self.member_id.overdue_loan_count > 0:
                return {
                    'warning': {
                        'title': '¡Atención!',
                        'message': f'El miembro
{self.member_id.partner_id.name} tiene '
f'{self.member_id.overdue_loan_count} préstamo(s) vencido(s).
',
                        f'Multas pendientes:
${self.member_id.pending_fees:.2f}'
                    }
                }

    @api.onchange('book_id')
    def _onchange_book_check_availability(self):
```

```

        """Verificar disponibilidad del libro al
        seleccionarlo"""
        if self.book_id:
            if self.book_id.available_qty <= 0:
                return {
                    'warning': {
                        'title': 'Libro No Disponible',
                        'message': f'El libro
"{self.book_id.name}" no tiene copias disponibles. '
                        f'Todas están prestadas
actualmente.'
                    }
                }
            elif self.book_id.available_qty <= 2:
                return {
                    'warning': {
                        'title': 'Pocas Copias Disponibles',
                        'message': f'Solo quedan
{self.book_id.available_qty} copias disponibles '
                        f'de "{self.book_id.name}".'
                    }
                }
    }

```

Verificación

1. Actualizar el módulo
2. Ir a Biblioteca → Préstamos → Crear
3. Seleccionar un miembro
4. Verificar que `expected_return_date` se llena automáticamente
5. Seleccionar un miembro con préstamos vencidos
6. Verificar que aparece advertencia
7. Seleccionar un libro sin disponibilidad
8. Verificar que aparece advertencia



EJERCICIO 5: Constraint - Validar Límite de Préstamos (INTERMEDIO-AVANZADO)

Conceptos: `@api.constrains`, validaciones, `ensure_one()`


```

        ('state', '=', 'confirmed'),
        ('id', '!=', loan.id)
    ])

    max_loans = loan.member_id.max_loans

    if active_loans >= max_loans:
        raise ValidationError(
            f'El miembro
{loan.member_id.partner_id.name} '
            f'ya tiene {active_loans} préstamo(s)
activo(s). '
            f'Su límite máximo es {max_loans}
préstamos simultáneos.\n\n'
            f'Tipo de membresía:
{dict(loan.member_id._fields["membership_type"].selection).get(
loan.member_id.membership_type)}'
        )

    def action_confirm(self):
        """
        Sobrescribir método para agregar validación adicional
        """
        self.ensure_one() # Asegurar que es singleton

        # Validar disponibilidad del libro
        if self.book_id.available_qty <= 0:
            raise ValidationError(
                f'El libro "{self.book_id.name}" no tiene
copias disponibles para préstamo.'
            )

        # Validar estado del miembro
        if self.member_id.state != 'active':
            raise ValidationError(
                f'El miembro {self.member_id.partner_id.name}
no está activo. '

```

```
        f'Estado actual:
{dict(self.member_id._fields["state"].selection).get(self.member_id.state)}'
    )

    # Validar multas pendientes
    if self.member_id.pending_fees > 0:
        raise ValidationError(
            f'El miembro {self.member_id.partner_id.name}
tiene multas pendientes de ${self.member_id.pending_fees:.2f}.
'
            f'Debe pagar las multas antes de realizar un
nuevo préstamo.'
        )

    # Llamar al método original
    return super().action_confirm()
```

Verificación

1. Actualizar el módulo
 2. Crear préstamos hasta alcanzar el límite de un miembro (ej: básico = 3)
 3. Intentar confirmar un préstamo adicional
 4. Verificar que aparece error de validación con mensaje claro
 5. Intentar confirmar préstamo con miembro suspendido
 6. Verificar error
 7. Intentar confirmar préstamo de libro sin disponibilidad
 8. Verificar error
-